

NRLC-02: NRQL → DQL Translation

Series: NRLC — New Relic to Dynatrace Migration Deep Dives | Notebook: 2 of 9 | Created: April 2026 | Last Updated: 10/05/2026

Overview

Query translation is the technical core of any New Relic → Dynatrace migration. NRQL is SQL-like and declarative; DQL is pipeline-based. A direct string substitution doesn't work — the structure, source resolution, and time semantics all differ. This deep dive covers the **architecture and behavior of the open-source `nrql-engine` compiler** (292 tested patterns, AST-based), the translation surface it covers, the confidence scoring it produces, the gaps it flags for manual review, and the patterns to recognize when reading its output.

The TypeScript `nrql-engine` and the Python compiler in `dynatrace-NewRelic/compiler/` are **pinned to each other via the Phase 19b regression suite** (`test_phase19b_engine_parity.py` in CI) — drift on either side trips the test. Both expose the same 292-pattern surface with identical confidence scoring.

Table of Contents

- 1. [Why a Compiler, Not Regex](#)
- 2. [Compiler Pipeline](#)
- 3. [Translation Surface](#)
- 4. [Confidence Scoring](#)
- 5. [Pattern Library — Worked Examples](#)
- 6. [Source-Class Mapping \(FROM clause\)](#)
- 7. [Function Mapping \(Aggregations\)](#)
- 8. [Time Expression Mapping](#)
- 9. [Unsupported NRQL — What to Do](#)
- 10. [Tooling: Engine, Translator, CLI](#)
- 11. [Validation Loop](#)

Prerequisites

Requirement	Details
Audience	Engineers translating NRQL queries; reviewers of automated translations

Requirement	Details
Recommended	NRLC-01 (platform comparison); familiarity with both NRQL and DQL syntax
Companion projects	nrql-engine , nrql-translator , NewRelic-to-Dynatrace-Migration-Utilities

Engine Support (Phase 19b)

The compiler is exposed through three projects (all pinned to the same 292-pattern surface via Phase 19b):

Surface	Module / CLI
Single NRQL query	<code>migrate.py compile "<nrql>"</code> or <code>nrql-translator query</code>
Batch translation	<code>migrate.py batch --file queries.csv</code> or <code>nrql-translator excel</code>
Translate + auto-fix + post-processing	<code>migrate.py convert --file queries.txt</code> (wraps compile + DQLFixer + Phase 19 apdex/compare/rate/percentage uplift)
Validate syntax only	<code>migrate.py compile --validate</code>
Shorthand pre-expand (<code>Distributed*</code> , <code>Mobile*</code> , <code>Lambda*</code> , etc.)	<code>compiler/shorthands.py</code> (9 patterns; Phase 19b)
DQL auto-fix rules	<code>validators/dql_fixer.py</code> (24 methods; Phase 19b one-to-one with TS <code>dql_fixer.ts</code>)

See §1 APM of the [coverage matrix \(migration utilities GitHub\)](#) for the full NRQL event-class → DT data-object mapping (Transaction, Span, Log, Metric, SystemSample, PageView, MobileSession, SyntheticCheck, and custom event types).

1. Why a Compiler, Not Regex

Early translation tools used regular expressions: find `SELECT count(*)` , replace with `summarize count = count()` . This breaks fast:

Failure mode	Why regex fails
Nested function calls	<code>percentage(count(*), WHERE status = 200)</code> requires recursion, not pattern match
<code>FACET CASES</code> with nested <code>if()</code>	Requires understanding the AST to emit nested DQL
Subqueries / <code>SELECT ... FROM (SELECT ...)</code>	Regex can't track scope
Time expressions	<code>SINCE 1 day ago UNTIL 1 hour ago</code> vs. <code>SINCE bin(now(), 24h)</code> need semantic awareness

Failure mode	Why regex fails
Compound <code>WHERE</code> with mixed operators	Operator precedence requires a parser

`nrql-engine` replaced an early 2,197-line regex translator with a proper compiler. The result: 292 tested translation patterns, **1,200+ unit tests in the Python orchestrator (post-Phase-24)**, and the ability to translate complex queries that the regex tool silently produced wrong DQL for. Phase 19b added a CI parity job (`test_phase19b_engine_parity.py`) that pins the Python compiler to the TS `nrql-engine` — drift on either side trips the test.

2. Compiler Pipeline

```

NRQL string
  ↓
  Lexer      → token stream
  ↓
  Parser     → AST (recursive descent)
  ↓
  Emitter    → DQL string + CompileResult
  ↓
  Validator  → syntactically valid? against tenant?
  ↓
  Auto-fixer → attempt safe corrections (operator normalization, duration literals)
  ↓
  Output: { dql, confidence, notes[], warnings[], fixes[] }

```

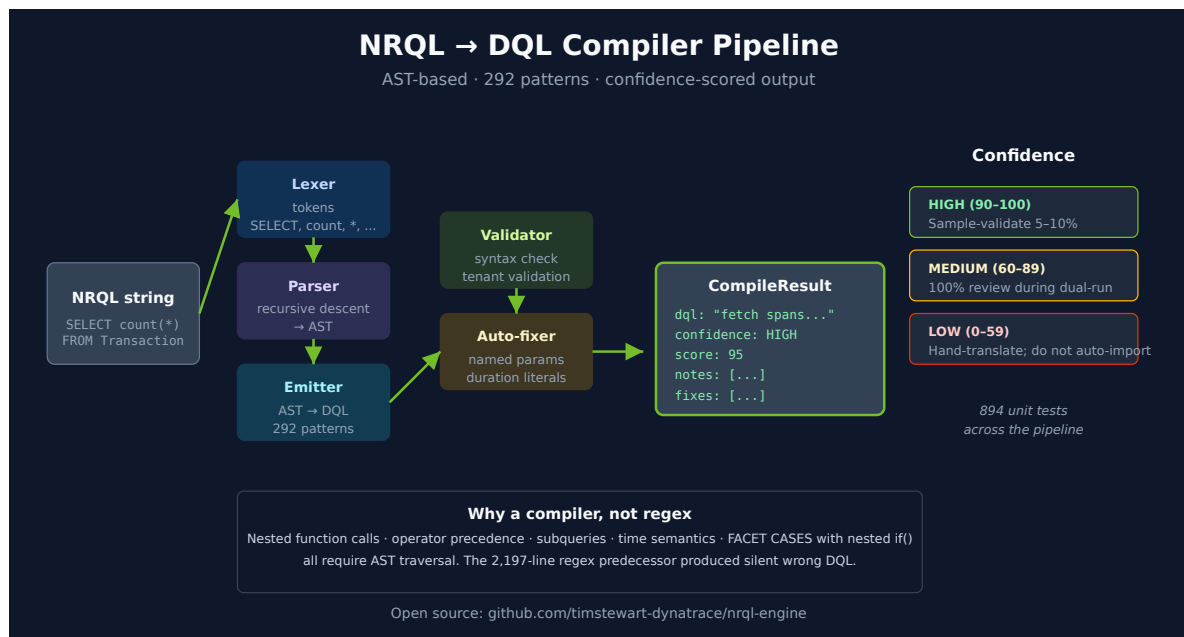
Each stage lives in a separate module: `compiler/lexer.py`, `compiler/parser.py`, `compiler/ast_nodes.py`, `compiler/emitter.py` (Python project), or `src/compiler/*.ts` (TypeScript engine).

The `CompileResult` is the contract every consumer uses:

```

interface CompileResult {
  success: boolean;
  dql: string;
  confidence: 'HIGH' | 'MEDIUM' | 'LOW';
  confidenceScore: number;    // 0-100
  ast?: NRQLNode;
  notes: TranslationNotes;    // structured guidance
  warnings: string[];
  fixes: string[];            // applied auto-fixes
}

```



3. Translation Surface

Supported NRQL clauses: SELECT, FROM, WHERE, FACET, TIMESERIES, LIMIT, ORDER BY, COMPARE WITH, SLIDE BY, SINCE, UNTIL

Supported aggregations: count, average / avg, sum, min, max, uniqueCount, percentile, latest, earliest, uniques, median, stddev, rate, percentage, cdfPercentage, filter, histogram, funnel, apdex (with caveats)

Supported expressions:

- Arithmetic between aggregations (sum(a) / count())
- Nested if() for conditional faceting
- Time grouping (hourOf, dateOf, weekOf)
- Subqueries via lookup patterns
- WHERE with AND / OR / NOT / IN / LIKE / regex

Partial / produces warnings:

- apdex() — emits DQL with TODO marker; threshold needs config
- bytecountestimate() — no DQL equivalent; recommend reformulation
- funnel() — multi-stage; emitted as nested append/join with manual review note

Not supported:

- cohort() — retention analysis is a different DT capability (Dynatrace Intelligence or custom DQL)
- Custom event types not present in DT (manual mapping required)
- Some metric streaming integrations — routed through OneAgent or OTLP instead

4. Confidence Scoring

Every translation gets a confidence label and a 0–100 score. The scoring is rule-based, not heuristic:

Confidence	Score	Meaning
HIGH	90–100	Pattern fully covered; structural and semantic equivalence
MEDIUM	60–89	Translation correct in common case; edge cases (NULLs, empty sets, time boundaries) need spot-check
LOW	0–59	Manual review required; one or more constructs lack a direct equivalent

What lowers confidence

Construct	Confidence Impact
Use of <code>apdex()</code>	LOW (threshold not portable)
Use of <code>funnel()</code>	LOW (multi-stage; manual review)
Subquery with <code>lookup</code>	MEDIUM (correctness depends on lookup data)
<code>SLIDE BY</code>	MEDIUM (DT supports but rarely-used path)
Nested <code>if()</code> in <code>FACET CASES</code>	MEDIUM (emits nested DQL; review structure)
<code>COMPARE WITH</code>	MEDIUM (emits append + time-shift; verify alignment)
Standard <code>SELECT count(*) FROM X WHERE Y FACET Z TIMESERIES</code>	HIGH

Acting on Confidence

- **HIGH** → trust the translation; sample-validate 5–10% during dual-run
- **MEDIUM** → 100% review during dual-run; verify edge cases
- **LOW** → hand-translate or reformulate the requirement; do not auto-import

5. Pattern Library — Worked Examples

Simple aggregation with FACET

NRQL:

```
SELECT count(*), average(duration) FROM Transaction
WHERE appName = 'checkout' SINCE 1 hour ago FACET host
```

DQL:

```
fetch spans, from:-1h
| filter dt.service.name == "checkout" and isNotNull(endpoint.name)
| summarize {count = count(), average_duration = avg(duration)}, by:{host.name}
```

Confidence: **HIGH (100)** — provided the translation keeps both filters. `isNotNull(endpoint.name)` restricts `fetch spans` to request root spans (one per NR `Transaction`); `dt.service.name` is the Dynatrace service name, whereas `service.name` is the OpenTelemetry resource attribute and misses spans that do not carry it. See §6.

Percentage with conditional

NRQL:

```
SELECT percentage(count(*), WHERE status = 200) FROM Transaction
```

DQL:

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name) and isNotNull(http.response.status_code)
| summarize success_pct = 100.0 * countIf(http.response.status_code == 200) / count()
```

Confidence: **HIGH (95)** only when the status field is mapped. Dynatrace spans carry the HTTP status as `http.response.status_code` (stable, type long) — map NR `httpResponseCode` / `http.statusCode` / `status` to it. A literal translation to `status == 200` or `http.status_code == 200` runs without error and returns 0% or close to it: neither field is in the semantic dictionary, and on the validation tenant, over request spans, they gave 0% and 13.5% where `http.response.status_code` gave 89.6% (10/05/2026).

Rate (decomposed)

NRQL:

```
SELECT rate(count(*), 1 minute) FROM Transaction
```

DQL:

```
fetch spans, from:-1h
| filter isNotNull(endpoint.name)
| makeTimeseries requests = count(), interval:1m
```

Confidence: **HIGH (90)** — NRQL `rate(count, N)` is structurally a per-interval count. Over event data DQL expresses it with `makeTimeseries`; `timeseries` reads stored metrics and needs a metric key (`timeseries count = count()` fails with *missing: metricKey*). For request counts, `timeseries requests = sum(dt.service.request.count, rollup: sum), interval:1m, from:-1h` reads the same rate from the service metric.

COMPARE WITH (time-shifted append)

NRQL:

```
SELECT count(*) FROM Transaction COMPARE WITH 1 week ago SINCE 1 day ago
```

DQL:

```

fetch spans, from:-1d
| filter isNotNull(endpoint.name)
| summarize current_count = count()
| append [
  fetch spans, from:-192h, to:-168h
  | filter isNotNull(endpoint.name)
  | summarize previous_count = count()
]

```

Confidence: **MEDIUM (75)** — verify the time alignment; DT default behavior on `append` may need a join for visualization.

Subquery via lookup

NRQL:

```

SELECT count(*) FROM PageView WHERE userAgent IN (SELECT name FROM BotList)

```

DQL:

```

fetch user.events, from:-1h
| filter characteristics.has_page_summary == true
| lookup [data record(name = "HeadlessChrome"), record(name = "Googlebot")],
  sourceField:browser.user_agent, lookupField:name, prefix:"bot."
| filter isNotNull(bot.name)
| summarize bot_page_views = count()

```

Confidence: **MEDIUM (70)** — correctness depends on where the bot list lives. NR page views are New RUM page summaries in `user.events`, not logs. The `data` subquery stands in for the bot list; replace it with your own table (for example a tabular file read with `load "/lookups/<file>"`). Confirm that `browser.user_agent` is populated on your real-user data first — on the validation tenant every page summary came from synthetic traffic and carried no user agent, so this query returned 0 (10/05/2026).

6. Source-Class Mapping (FROM clause)

NRQL's `FROM` references NR event classes; DQL's `fetch` references Grail data objects.

NRQL FROM	DQL fetch	Notes
Transaction	<code>fetch spans + filter isNotNull(endpoint.name)</code>	One record per request: <code>endpoint.name</code> is set only on request root spans. <code>span.kind == "server"</code> over-counts (it also matches server spans that are not request roots). Map <code>appName</code> to <code>dt.service.name</code> , not the OpenTelemetry <code>service.name</code>

NRQL FROM	DQL fetch	Notes
TransactionError	<code>fetch spans + filter isNotNull(endpoint.name) and span.status_code == "error"</code> , or the <code>dt.service.request.failure_count</code> metric	<code>error</code> is not a Dynatrace field — <code>isNotNull(error)</code> matches only spans where some instrumentation set a custom <code>error</code> attribute, and read 0 failures on a service with 235 of 749 requests failing in five minutes (10/05/2026)
Span	<code>fetch spans</code>	Direct
Log	<code>fetch logs</code>	Direct
Metric	<code>timeseries <agg>(<metric.key>)</code>	NRQL metrics → DT metrics
SystemSample	<code>timeseries avg(dt.host.cpu.usage)</code> (etc.)	Infrastructure host metrics
ProcessSample	<code>timeseries ... by: {dt.entity.process_group_instance}</code>	Process metrics
K8sContainerSample	<code>timeseries ... by: {k8s.container.name}</code>	K8s container metrics
PageView	<code>fetch user.events + filter characteristics.has_page_summary == true</code>	New RUM page summaries; RUM data is not in <code>logs</code> or <code>events</code>
BrowserInteraction	<code>fetch user.events + the matching characteristics.has_* flag (for example has_user_action)</code>	New RUM; see WEBRUM-09 for the event characteristics
MobileSession	<code>fetch user.sessions</code>	New RUM sessions
SyntheticCheck	<code>fetch dt.synthetic.events + filter in(event.type, {"http_monitor_execution", "browser_monitor_execution"})</code>	For availability, also <code>filter not(coalesce(execution.retry_on_error, false) and result.state == "FAIL")</code> — a retried browser failure is otherwise counted twice
Custom event type	<code>fetch bizevents</code> or <code>fetch events</code>	Depends on ingest mapping

The compiler picks the `fetch` source from a built-in lookup. There is no CLI flag to override it — if your NR custom event lands in a different DT data object, edit the translated DQL.

7. Function Mapping (Aggregations)

NRQL	DQL	Notes
<code>count(*)</code>	<code>count()</code>	
<code>count(field)</code>	<code>count(field)</code>	counts non-null
<code>uniqueCount(field)</code>	<code>countDistinct(field)</code>	

NRQL	DQL	Notes
<code>average(field) / avg(field)</code>	<code>avg(field)</code>	
<code>sum(field)</code>	<code>sum(field)</code>	
<code>min(field) / max(field)</code>	<code>min(field) / max(field)</code>	
<code>percentile(field, 95)</code>	<code>percentile(field, 95)</code>	DQL uses positional; NRQL uses positional
<code>median(field)</code>	<code>median(field)</code>	
<code>stddev(field)</code>	<code>stddev(field)</code>	on a duration field use <code>stddev(duration / 1ms)</code> — <code>stddev(duration)</code> returns 0 with no warning
<code>latest(field)</code>	<code>takeLast(field)</code>	with <code>sort timestamp desc</code> if needed
<code>earliest(field)</code>	<code>takeFirst(field)</code>	
<code>uniques(field)</code>	<code>collectDistinct(field)</code>	
<code>rate(count(*), 1 minute)</code>	<code>makeTimeseries count(), interval:1m</code>	structural rewrite; <code>timeseries</code> needs a metric key
<code>percentage(count(*), WHERE x)</code>	<code>100.0 * countIf(x) / count()</code>	
<code>filter(count(*), WHERE x)</code>	<code>countIf(x)</code>	
<code>histogram(field, ...)</code>	<code>summarize ..., by: {bin(field, N)}</code>	bucket-based; on a duration use a duration width, e.g. <code>bin(duration, 100ms)</code> (bin has no <code>width: parameter</code>)
<code>apdex(field, t:N)</code>	flagged TODO; manual config	DT uses Apdex via Dynatrace Intelligence or custom DQL
<code>funnel(...)</code>	flagged; nested append/join skeleton	manual review
<code>cohort(...)</code>	unsupported	use Dynatrace Intelligence or custom DQL

8. Time Expression Mapping

NRQL	DQL
<code>SINCE 1 hour ago</code>	<code>from:-1h</code>
<code>SINCE 1 day ago UNTIL 1 hour ago</code>	<code>from:-1d, to:-1h</code>

NRQL	DQL
<code>SINCE '2026-04-14 00:00:00'</code>	<code>from:"2026-04-14T00:00:00Z"</code>
<code>TIMESERIES 5 minutes</code>	<code>interval:5m</code>
<code>TIMESERIES MAX</code>	omit <code>interval:</code> — <code>makeTimeseries</code> / <code>timeseries</code> choose the interval automatically (<code>interval: auto</code> does not parse)
<code>SLIDE BY 1 minute</code>	not directly supported — use <code>bin(timestamp, 1m)</code> for similar shape
<code>COMPARE WITH 1 week ago</code>	second <code>fetch</code> with <code>from:-2w, to:-1w</code> plus shift

The compiler normalizes all time expressions to absolute or relative DQL forms. A common gotcha: NRQL's default time window (1 hour) differs from DQL's defaults; the compiler always emits an explicit `from:` to avoid this.

9. Unsupported NRQL — What to Do

When the compiler emits LOW confidence or a TODO marker:

Symptom	Cause	Action
Translation contains <code>// TODO: APDEX_THRESHOLD</code>	NRQL <code>apdex(field, t:0.5)</code> — threshold doesn't translate	Configure DT Apdex calculation separately or use Dynatrace Intelligence SLO with target value
Translation contains <code>// TODO: FUNNEL_STAGES</code>	Multi-stage funnel	Emit per-stage metrics + custom DQL with <code>lookup chain</code>
Compiler returns <code>success: false</code>	Unparseable NRQL (likely malformed or uses a private NR feature)	Hand-translate; capture in gap-analysis
LOW confidence on subquery	<code>lookup target</code> not validated against Grail	Check that the looked-up data exists as queryable Grail content
LOW confidence on custom event type	Source mapping unknown	Add a source mapping override or route the data via OpenPipeline first

Document gaps in the migration's gap-analysis artifact (NR2DT-01 Discover §6). LOW-confidence queries are a planned cost, not a surprise.

10. Tooling: Engine, Translator, CLI

Three projects expose the same compiler in different shapes:

Tool	Form	Use Case
nrql-engine	TypeScript library (<code>npm install @timstewart-dynatrace/nrql-engine</code>)	Embed in your own tool, web app, or CI — planned future home: dynatrace-dma (Dynatrace Migration Assistant)
nrql-translator	Thin CLI around the engine	Single query, batch Excel, validation, notebook generation
NewRelic-to-Dynatrace-Migration-Utilities	Python CLI orchestrating full migration	End-to-end NR → DT including all entity transformers

Single-query CLI

```
# Using nrql-translator
nrql-translator query "SELECT count(*) FROM Transaction FACET host"

# Or Dynatrace-NewRelic Python CLI
python3 migrate.py compile "SELECT count(*) FROM Transaction FACET host"
```

Batch translation

```
# CSV in, CSV out (with confidence column)
python3 migrate.py batch --file all-queries.csv

# Excel batch (preserves dashboard mapping)
nrql-translator excel --in dashboards.xlsx --out dashboards-dql.xlsx
```

Notebook output

The translator can emit a Dynatrace notebook directly from a batch:

```
nrql-translator notebook --in dashboard-queries.csv --out migrated-dashboard.ipynb
```

Dynatrace Migration Assistant ([dynatrace-dma](#)) Home

The [dynatrace-dma](#) GitHub organization is the upstream **Dynatrace Migration Assistant** home for source-platform migration tooling. It already hosts:

- [splunk-to-dynatrace](#) — Splunk → Dynatrace migration resources
- [datadog-to-dynatrace](#) — Datadog configuration extraction scripts

The NRQL engine is planned to relocate here under a [newrelic-to-dynatrace](#) repo following the same pattern. Until the move, the engine remains canonical at [timstewart-dynatrace/nrql-engine](#) ; track [dynatrace-dma](#) for the official Dynatrace-supported version going forward.

11. Validation Loop

Translation alone doesn't guarantee correctness. The full loop is:

```
NRQL → [compile] → DQL → [syntax validate] → [tenant validate] → [behavioral validate]
```

1. **Syntax validate** — the engine's `DQLSyntaxValidator` posts the DQL to the Grail language service's `POST /platform/storage/query/v1/query:verify` endpoint for parser-level validation. No data executes.
2. **Tenant validate** — `DTEnvironmentRegistry` checks that referenced metrics, entities, buckets, and OpenPipeline enrichment attributes actually exist in the target tenant.
3. **Behavioral validate** — run both NRQL (against NR) and translated DQL (against DT) over the same time window and compare result shape and magnitudes.

Auto-fix

When syntax validation fails, the engine attempts safe corrections before reporting failure:

- Operator normalization (`>` becomes `>=` if NRQL was inclusive)
- Duration literal correction (`1m` → `1m` ensures `m` suffix lowercase)
- Field name normalization (camelCase ↔ snake_case for known mappings)

Applied fixes are returned in `result.fixes[]` so reviewers can see what changed.

Summary

NRQL → DQL translation is a compiler problem, not a string-replace problem. The open-source `nrql-engine` covers 292 patterns with confidence scoring; LOW-confidence queries are flagged for manual review rather than silently emitted as wrong DQL. Pair the translator with the validation loop, treat MEDIUM confidence as a review obligation, and keep an explicit gap-analysis for the genuinely unsupported constructs.

Continue to **NRLC-03 Dashboard Migration** to see how translated queries flow into Dynatrace dashboard artifacts.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source [NewRelic-to-Dynatrace-Migration-Utilities](#), [nrql-engine](#) (planned future home: the [dynatrace-dma](#) Dynatrace Migration Assistant organization), and [nrql-translator](#) projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official [Dynatrace documentation](#) and [New Relic documentation](#).